

Curtin University
Cyber Crime and Security
Enhanced Programming
ISEC3004
Group 33

Dilshodbek Niyazimbetov: 21850529

S M Ihtesham Tahmid: 21711367

Jun Ching Chai: 22978723

Gerard Vincent Cabriles: 20357223

Table of Contents

1 Introduction.....	3
1.1 Project Background.....	3
1.2 Project Objectives.....	3
1.3 Selected Vulnerabilities.....	4
1.4 Scope and Limitations.....	4
2 Vulnerability 1 - OAuth Session Fixation.....	5
2.1 Overview.....	5
2.2 Significance.....	5
2.3 Vulnerable Code.....	5
2.4 Code Exploitation.....	7
2.5 Impact & Consequences.....	9
2.6 Detection & Tracing.....	10
2.7 Mitigation (Security Enhanced Code).....	10
2.8 Verification Of Mitigation.....	11
3 Vulnerability 2 - Cross Site Request Forgery (CSRF).....	13
3.1 Overview.....	13
3.2 Significance.....	14
3.3 Vulnerable Code.....	14
3.4 Code Exploitation.....	15
3.5 Impact & Consequences.....	15
3.6 Detection & Tracing.....	16
3.7 Mitigation (Security Enhanced Code).....	16
3.8 Verification Of Mitigation.....	16
4 Evidence of Project Management.....	17
4.1 Task Allocation.....	17
4.2 Effort Estimation and Scheduling.....	17
4.3 Progress Monitoring.....	18
4.4 Quality Assurance.....	20
5 Discussions & Recommendations.....	20
5.1 Comparison of Mitigation Strategies.....	20
5.2 Security Recommendations.....	21
6 Conclusion.....	22
7 References.....	23
8 Appendix.....	23

1 Introduction

1.1 Project Background

Security vulnerabilities are prevalent when web applications incorrectly implement authentication, session management or request validation.

- While an application may successfully authenticate a legitimate user, weakness in the surrounding mechanisms can still allow an attacker to misuse the authenticated state.

The purpose of this project is to have a practical investigation of these issues rather than only examining the vulnerabilities only from a theoretical perspective.

- The following project uses vulnerable code to reproduce security weaknesses that may be encountered in many applications, followed by a controlled exploitation to demonstrate the potential consequences that may be faced if remained unresolved.
- Evidence obtained from the simulated attacks is then collected and analysed to identify how malicious activity may be detected and traced.

After demonstrating each vulnerability, a new version of the demo is then implemented with mitigations applied to them. The effectiveness of selected mitigation techniques is then evaluated by attempting to reproduce original attacks against the new and modified demo.

1.2 Project Objectives

The primary objective of this project is to develop a practical understanding of web application vulnerabilities and the techniques used to exploit, detect and mitigate them. The project aims to:

- Investigate operation and security significance of OAuth Session Fixation and CSRF.
- Develop a vulnerable application deliberately to demonstrate the security weaknesses associated with each vulnerability.
- Reproduce and document the exploitation process for each vulnerability in a controlled environment.
- Examine the potential impact and consequences of successful exploitation.
- Collect and analyse application logs and other evidence generated during the attacks.
- Implement appropriate security controls to mitigate the identified vulnerabilities.
- Repeat the attack against the updated applications with the security implementations to verify the effectiveness of the implemented mitigations.

1.3 Selected Vulnerabilities

The two vulnerabilities selected by our group for investigation are:

- OAuth Session Fixation; and
- Cross Site Request Forgery (CSRF).

1.4 Scope and Limitations

The scope of this project is limited to controlled demonstrations of OAuth Session Fixation and CSRF using applications developed specifically for security testing.

- For each vulnerability, the project examines the vulnerable implementation, exploitation procedure, the resulting security impact, detection and tracing evidence, the mitigation technique and verification of the updated code with the enhanced implementation.

The demonstrations are conducted within a controlled and local environment and are intended to reproduce the fundamental behaviour of each vulnerability.

- The project does not attempt to target or exploit external production systems.
- The applications are purposely simplified so that the vulnerable behaviour and corresponding mitigation techniques are the only one being demonstrated.

Thus, the demonstrations do not represent every possible implementation or exploitation technique associated with OAuth Session Fixation or Cross Site Request Forgery (CSRF).

- Real world examples may use additional security mechanisms that affect how these attacks can be performed or detected.
- The results of this project should only be interpreted as demonstrations of the security concepts instead of a complete representation of all real world attack scenarios.

2 Vulnerability 1 - OAuth Session Fixation

2.1 Overview

Session Fixation is a web application vulnerability that lets an attacker steal a valid user session by exploiting a failure in the session management identifier.

The main flaw for Session Fixation is straightforward:

- When a user attempts to authenticate, the application fails to assign a new session ID. This means an attacker who knows or controls that session identifier can use it to access an already authenticated session as if they were a legitimate user.
- This attack is not a type of Session Hijacking, which steals the established session between the client and the Web Server after the user logs in. Instead, it establishes a session on the victim's own browser, so the attack starts before the user even logs on.

2.2 Significance

There are 4 reasons why Session Fixation is a significant issue to address:

- First, it affects the relying application rather than the identity provider, meaning the vulnerability itself can exist in trusted providers like Google, which is used for login. So developers assume that the provider's security is guaranteed, which is incorrect.
- Second, it requires low attacker sophistication – it can be achieved through something as simple as sending a well-crafted custom link that looks legitimate.
- Third, a successful attack will result in full account takeover rather than a partial one.
- Finally, this attack is well documented but oftentimes overlooked as a vulnerability.

2.3 Vulnerable Code

The vulnerable code is a small Node.js web app which acts as an OAuth relying party.

- This website basically allows users to sign in using an external identity provider rather than managing passwords itself.
- The login provider is a local mock OAuth server that stands in for a real provider and the user authenticates there using a demo account alice / password123.

The system runs as 3 cooperating processes on localhost:

- Mock Provider (Port 7000)
- Vulnerable Relying Party (Port 7001)
- Attacker Controlled Site (Port 7050)

The vulnerability resides in how the relying party manages its own session.

The application contains 2 related vulnerabilities that together enable session fixation.

VULNERABILITY 01

The session identifier can be set from the URL.

- A middleware adopts any session ID supplied as a query parameter, which allows attacker to pin session ID they already know onto victim's browser before the victim logs in.

```
// -----  
// VULNERABLE fixation sink: honour a session id supplied in the URL.  
// A real app might do this via a "session in the URL" misconfiguration or a  
// permissive cookie. It lets an attacker pin a known session ID onto the  
// victim's browser before authentication.  
app.use((req, res, next) => {  
  if (req.query.sid) {  
    req.headers.cookie = `sid=${req.query.sid}`;  
    res.setHeader('Set-Cookie', `sid=${req.query.sid}; Path=/`);  
    log(`FIXATION: session id taken from URL -> sid=${req.query.sid}`);  
  }  
  next();  
});  
// -----
```

VULNERABILITY 02

The session identifier is not regenerated at login.

- When the OAuth callback completes successfully, the application marks the current session as authenticated but leaves its ID unchanged.
- Whatever session ID browser arrived with will become a fully authenticated session. The returned OAuth state value is also ignored rather than verified.

```
// OAuth callback: exchange the code and mark THIS session authenticated.
app.get('/callback', async (req, res) => {
  const { code, state } = req.query;
  if (!code) return res.status(400).send('missing code');

  // VULNERABLE: the returned `state` is ignored -- no verification at all.
  const tokenRes = await fetch(`${PROVIDER}/token`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
    body: `code=${encodeURIComponent(code)}`,
  });
  const token = await tokenRes.json();
  if (!token.user) return res.status(401).send('token exchange failed');

  // -----
  // VULNERABLE: the session is marked authenticated but its ID is left
  // unchanged. Whatever session ID the browser arrived with (possibly one an
  // attacker fixated via ?sid=) is now a fully authenticated session.
  req.session.authenticated = true;
  req.session.user = token.user.name;
  req.session.email = token.user.email;
  // -----

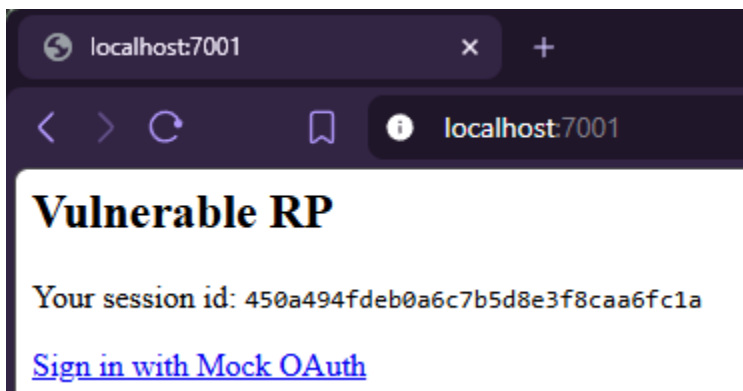
  log(`LOGIN SUCCESS (no session regeneration) | sid=${req.sessionID} | user=${token.user.name} | state_returned=${state}`);
  res.redirect('/account');
});
```

The risk introduced by this code is full account takeover without any credential theft.

- Because pre-login session ID survives authentication, an attacker who knows that ID holds a valid, authenticated session the moment the victim signs in, even though the attacker never saw the victim's password.
- Reusing any fixed session ID lets the attacker access victim's authenticated/account page as if they are the legitimate user.

2.4 Code Exploitation

First, the attacker will visit the OAuth web application.



The attacker will then include the session ID in the link and send that to the victim.

http://localhost:7001/?sid=450a494fdeb0a6c7b5d8e3f8caa6fc1a

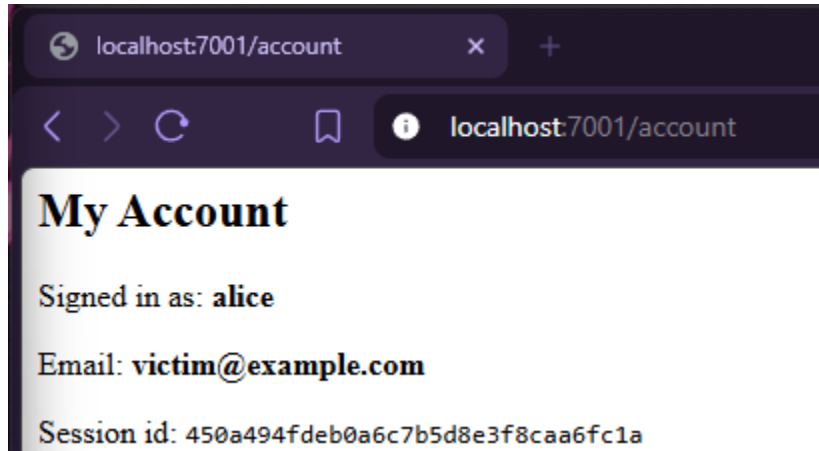
Let's say the victim had already clicked on the link and decided to login as usual.

Mock OAuth Provider — Sign in

Application **vuln-rp** wants to sign you in.

(demo credentials pre-filled: alice / password123)

It will then show the login details and current session ID as below.

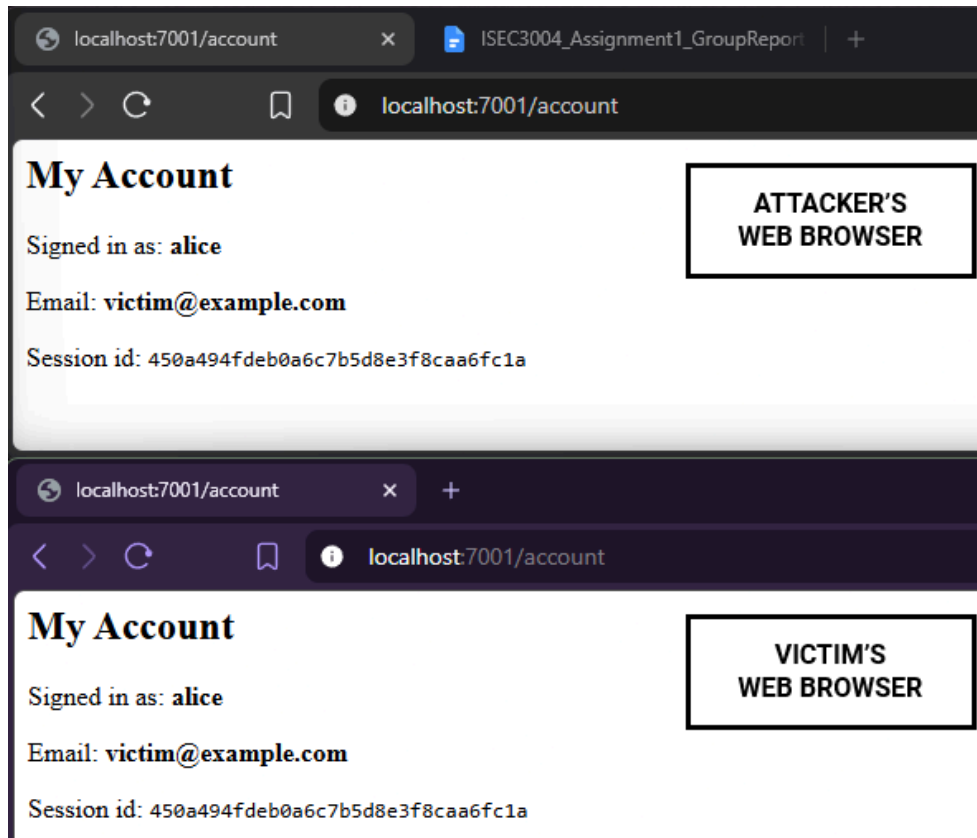


After that is done, the attacker will use the following link to steal personal details.

<http://localhost:7001/account>

So now, the attacker will then have the details for that particular session ID.

- All of these are done without the attacker logging in, relying on the victim to login.
- The attacker just needs to craft a session ID and the victim using that ID in order to work.
- They do not need to share the same cookie or browser or computer.



The image above shows that the attacker is able to access the victim's personal details.

2.5 Impact & Consequences

The root cause is because of the web application's flaw.

- It does not regenerate the session ID after authentication, nor does it check for the current session ID is legitimate or not before accepting it.
- The legitimate looking link given by the attacker has put in a session ID that the attacker uses to track down personal details. The attacker can use the current session that is logged in by the victim to change password and lock the user out.
- With the current session, the attacker can also steal sensitive information that can be used to steal money or hack into other accounts to exploit them too.

2.6 Detection & Tracing

```
[2026-10-06T06:05:08.535Z] Request GET / | sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | origin=none | referer=none | authenticated=false
[2026-10-06T06:07:23.583Z] FIXATION: session id taken from URL -> sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1
[2026-10-06T06:07:23.585Z] Request GET / | sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | origin=none | referer=none | authenticated=false
[2026-10-06T06:08:16.441Z] Request GET /login | sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | origin=none | referer=http://localhost:7001/?sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | authenticated=false
[2026-10-06T06:08:16.443Z] Starting OAuth | sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | state=m3d3yzhkbjk (not bound to session)
[2026-10-06T06:08:17.686Z] Request GET /callback | sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | origin=none | referer=http://localhost:7000/ | authenticated=false
[2026-10-06T06:08:17.693Z] LOGIN SUCCESS (no session regeneration) | sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | user=alice | state_returned=m3d3yzhkbjk
[2026-10-06T06:08:17.697Z] Request GET /account | sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | origin=none | referer=http://localhost:7000/ | authenticated=true
[2026-10-06T06:08:17.697Z] ACCOUNT VIEWED | sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | user=alice
[2026-10-06T06:09:15.413Z] Request GET /account | sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | origin=none | referer=none | authenticated=true
[2026-10-06T06:09:15.415Z] ACCOUNT VIEWED | sid=450a494fdeb0a6c7b5d8e3f8caa6fcl1 | user=alice
```

From the console, you can see the following actions that had been logged.

- First, the system receives the following session ID as used in section 2.4 of this report.
- Then, another browser (victim's) clicks on the link with session ID and request to login.
- After logging in, the system will then start OAuth and immediately approve the login details. As you can see in the image above, there is no session ID regeneration done.
- The first **ACCOUNT VIEWED** shows the victim viewing his/her own account details.
- Lastly, the hacker accessed the account website directly and also viewed the victim's account details, as shown in the second **ACCOUNT VIEWED** section.

2.7 Mitigation (Security Enhanced Code)

First, we modify the code so that when the user is authenticated, it will move into the new session ID and destroy the old session ID.

```
// -----
// MITIGATED: regenerate the session ID now that the user is authenticated.
// The old ID (which an attacker may have fixated) is destroyed and the
// browser is given a fresh, server-chosen ID the attacker never saw.
const { oldId, newId } = regenerateSession(req, res, COOKIE_FLAGS);
// -----
```

Second, we have to make sure that the old session ID cannot be used to access someone's personal information - in this case, we have to verify the state parameter matches.

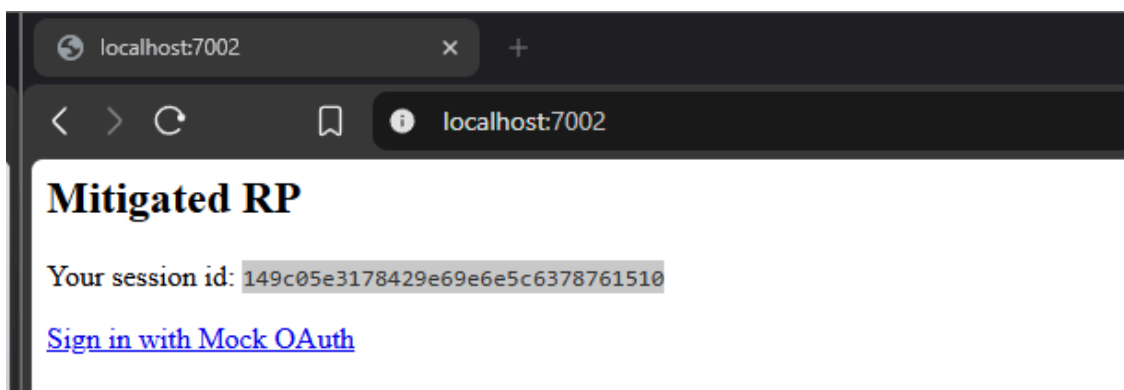
```
// -----  
// MITIGATED: verify the returned state matches the one tied to this session.  
if (!state || state !== req.session.oauthState) {  
  log('BLOCKED: state mismatch on callback | sid=${req.sessionID} | expected=${req.session.oauthState} | got=${state}');  
  return res.status(403).send('Forbidden: invalid OAuth state');  
}  
req.session.oauthState = null; // one-time use  
// -----
```

Finally, we make sure that the session ID is considered as system-generated and only linked to the exact same browser where the session ID is generated.

```
// -----  
// MITIGATED: generate a state value and bind it to THIS session so the  
// callback can confirm the response belongs to the flow this browser began.  
const state = randomId();  
req.session.oauthState = state;  
// -----  
const url = `${PROVIDER}/authorize?response_type=code&client_id=secure-rp`  
  + `&redirect_uri=${encodeURIComponent(REDIRECT_URI)}&state=${state}`;  
log(`Starting OAuth | sid=${req.sessionID} | state=${state} (bound to session)`);  
res.redirect(url);  
  
// MITIGATED: no ?sid= fixation sink here -- the session ID comes only from the  
// server-issued cookie, which is HttpOnly + SameSite=Lax.  
app.use(createSessionMiddleware(COOKIE_FLAGS));
```

2.8 Verification Of Mitigation

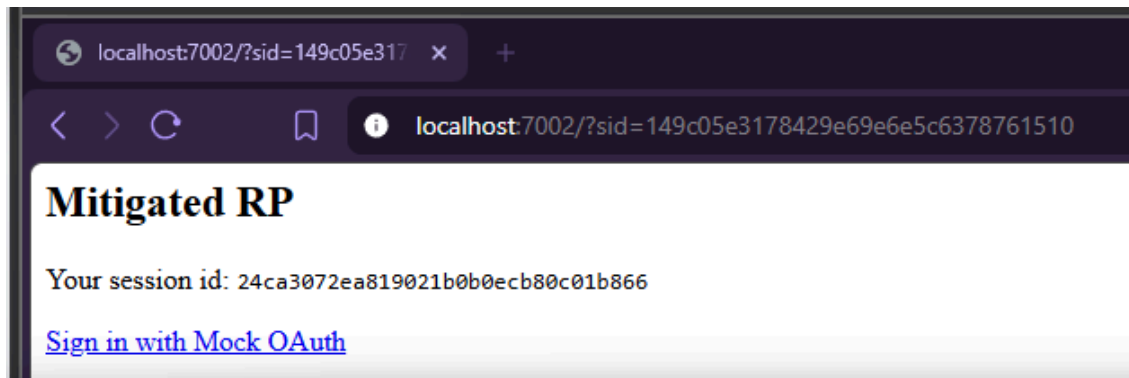
To verify the mitigation, the attacker will visit the OAuth web application first.



The attacker will copy the session ID and send that link to the victim.

http://localhost:7002/?sid=149c05e3178429e69e6e5c6378761510

The victim will then receive the link through a legitimate looking email/message. However, when the victim clicked on the link, the image below will be seen.



Observe that the website shows a completely different session ID than the link's session ID.

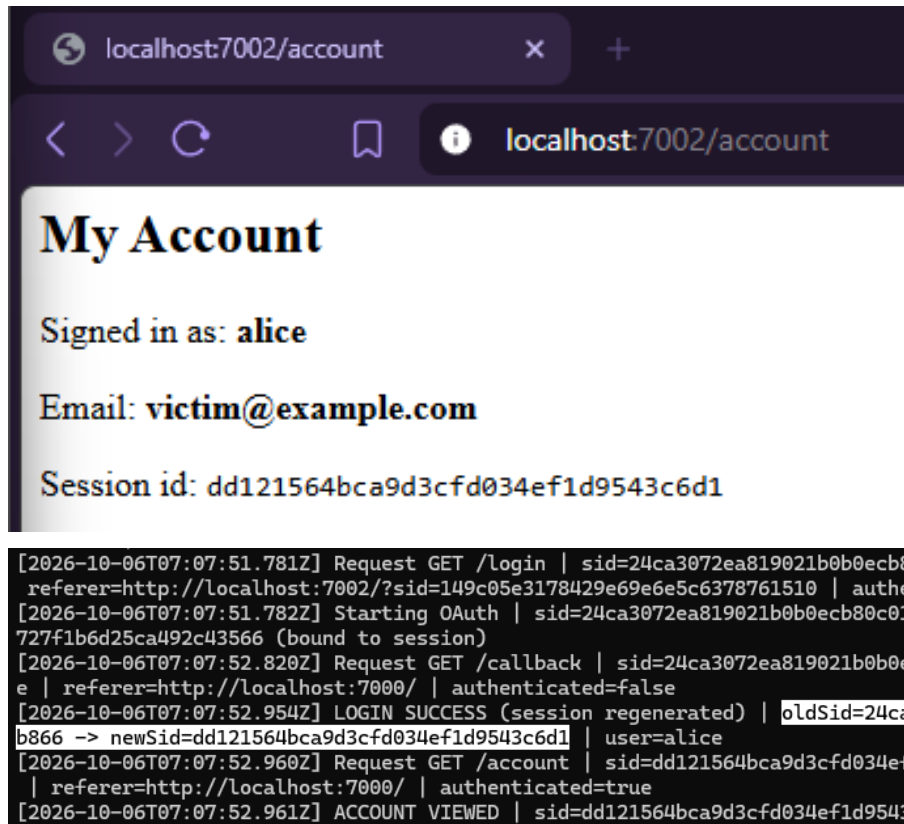
- This is because the system has regenerated the current session ID to make sure the attacker cannot use session ID to exploit and extract personal information.

```
Command Prompt - node pro X Command Prompt - node mi X + v
Microsoft Windows [Version 10.0.28120.3151]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>cd desktop/oauth-demo

C:\Users\user\Desktop\oauth-demo>node mitigated-app.js
[2026-10-06T07:04:36.077Z] MITIGATED OAuth RP listening on http://localhost:7002
[2026-10-06T07:05:04.637Z] Request GET / | sid=149c05e3178429e69e6e5c6378761510 | origin=none | referer=none | authenticated=false
[2026-10-06T07:05:18.312Z] Request GET / | sid=24ca3072ea819021b0b0ecb80c01b866 | origin=none | referer=none | authenticated=false
```

To prove that the session ID always regenerates, the victim can login and check out the new session ID - which is also different from the first session ID that had been replaced.



In the image above, session ID **dd121564bca9d3cfd034ef1d9543c6d1** is not the same as session ID **24ca3072ea819021b0b0ecb80c01b866** as it changes after logging in.

3 Vulnerability 2 - Cross Site Request Forgery (CSRF)

3.1 Overview

Cross Site Request Forgery (CSRF) is an attack in which a website, email or program causes an already authenticated user's browser to submit a request to a trusted application without users knowledge or intent, tricking the authenticated user's browser into performing an unwanted action on a trusted website.

- The vulnerability exists because once a user is authenticated to a site, the site has no way to distinguish between a legit request and a forged one.
- This attack specifically targets state-changing requests - actions such as changing credentials, transferring funds, or modifying accounts settings on the user's behalf.

3.2 Significance

CSRF is significant because it undermines the core assumption that an authenticated session implies an intentional request. It is a widely misunderstood vulnerability that has repeatedly been featured in the OWASP Top 10 list, and is formally tracked as CWE-352.

The practical reasons why this vulnerability is significant:

- Low attack effort, high impact - the attacker doesn't need to steal credentials, they only need the victim to load a page while already logged in elsewhere.
- A successful attack exploit can compromise end-user data.

There have been stories of this attack attacking home routers by changing DNS settings.

3.3 Vulnerable Code

This is an application with the main flaw, which is that the server relies only on the session cookie to authorise the request, without CSRF token or verification. This means any website the victim's browser visits can submit this form on their behalf.

```
app.post('/account/update-email', (req, res) => {  
  if (!req.session.authenticated) return res.status(401).send('Not logged in');  
  
  // -----  
  // VULNERABLE: relies only on the session cookie for authorization.  
  // No CSRF token check, no Origin/Referer verification -- any site  
  // the victim's browser visits can submit this form on their behalf.  
  const { newEmail } = req.body;  
  req.session.email = newEmail;  
  // -----  
  
  log(`EMAIL CHANGED to ${newEmail} | sid=${req.sessionID} | origin=${req.headers.origin || 'none'} | referer=${req.headers.referer || 'none'}`);  
  res.send(`Email updated to ${newEmail}`);  
});
```

this is part which we will use to change user's email

```
<h1>You won a prize! Claim it now...</h1>  
<form action="http://localhost:4000/account/update-email" method="POST" id="csrf-form">  
  <input type="hidden" name="newEmail" value="attacker@evil.com">  
</form>  
<script>  
  document.getElementById('csrf-form').submit();  
</script>
```

example code of the attacker's page

3.4 Code Exploitation

First, the user writes their username and password to log in.

Login

alice	password123	Log in
------------------------------------	--	---------------------------------------

(demo credentials pre-filled: alice / password123)

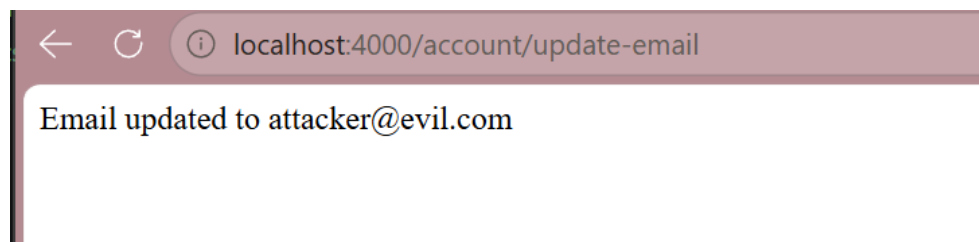
Then, the user is successfully authenticated.

My Account

Current email: **victim@example.com**

new email	Update email
--	---

After authentication, the user “decided” to visit attacker’s page.



After going back to the account page, the user’s email has been changed.

My Account

Current email: **attacker@evil.com**

new email	Update email
--	---

3.5 Impact & Consequences

The above example is the act of an attacker silently redirecting the account recovery (changing email is often the first step to a full account takeover via forgot password), with no interaction beyond visiting a webpage.

3.6 Detection & Tracing

```
[2026-10-02T05:58:53.113Z] Request POST /login | sid=30e7c16cb737f13d8cc6dce2adf22ce7 | origin=http://localhost:4000 | referer=http://localhost:4000/login | authenticated=false
[2026-10-02T05:58:53.114Z] Login success for user=alice | sid=30e7c16cb737f13d8cc6dce2adf22ce7
[2026-10-02T05:58:53.118Z] Request GET /account | sid=30e7c16cb737f13d8cc6dce2adf22ce7 | origin=none | referer=http://localhost:4000/login | authenticated=true
[2026-10-02T05:59:18.841Z] Request POST /account/update-email | sid=30e7c16cb737f13d8cc6dce2adf22ce7 | origin=http://localhost:5050 | referer=http://localhost:5050/ | authenticated=true
[2026-10-02T05:59:18.842Z] EMAIL CHANGED to attacker@evil.com | sid=30e7c16cb737f13d8cc6dce2adf22ce7 | origin=http://localhost:5050 | referer=http://localhost:5050/ | authenticated=true
[2026-10-02T05:59:18.843Z] VULNERABLE CSRF: req.session.req.headers['origin']=http://localhost:4000
```

example of logs - shows that request itself didn't originate from the application's own page

3.7 Mitigation (Security Enhanced Code)

```
app.post('/account/update-email', (req, res) => {
  if (!req.session.authenticated) return res.status(401).send('Not logged in');

  // -----
  // MITIGATED: verify the submitted CSRF token matches the one tied
  // to this session before performing the state-changing action. A
  // cross-site attacker cannot read the victim's page to obtain this
  // token (same-origin policy), so a forged request will fail here.
  const submittedToken = req.body.csrf_token;
  if (!submittedToken || submittedToken !== req.session.csrfToken) {
    log('BLOCKED: invalid/missing CSRF token on update-email | sid=${req.sessionID} | origin=${req.headers.origin || 'none'} | referer=${req.headers.referer || 'none'}');
    return res.status(403).send('Forbidden: invalid CSRF token');
  }
  // -----

  const { newEmail } = req.body;
  req.session.email = newEmail;
  log('EMAIL CHANGED to ${newEmail} | sid=${req.sessionID} | csrf token verified');
  res.send('Email updated to ${newEmail}');
});
```

The image above shows the code to fix the vulnerability.

- The token is generated server-side per session and only exists inside the accounts pages' HTML. So in return, the attacker's page can not read that token to include it.

3.8 Verification Of Mitigation

The user is authenticated.

My Account

Current email: **victim@example.com**

```
HTTP/1.1 403 Forbidden
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 29
ETag: W/"1d-CoF7jhOnTWggo+Gj2ubPJcPySKA"
Date: Fri, 02 Oct 2026 06:50:19 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Rerunning the identical cross origin attack resulted in the 403 Forbidden Response error and no change in the victims account.

```

MITIGATED CSRF app listening on http://localhost:4001
Request GET /login | sid=e9eb7b4fa03171898e53ce7901b39bd8 | origin=none | referer=none | authenticated=false
Request GET /favicon.ico | sid=e9eb7b4fa03171898e53ce7901b39bd8 | origin=none | referer=http://localhost:4001/login | authenticated=false
Request POST /login | sid=e9eb7b4fa03171898e53ce7901b39bd8 | origin=http://localhost:4001 | referer=http://localhost:4001/login | authenticated=false
Login success for user=alice | sid=e9eb7b4fa03171898e53ce7901b39bd8
Request GET /account | sid=e9eb7b4fa03171898e53ce7901b39bd8 | origin=none | referer=http://localhost:4001/login | authenticated=true
Request POST /login | sid=d910ed5ec0e3fbfcd309be400460d63a | origin=none | referer=none | authenticated=false
Login success for user=alice | sid=d910ed5ec0e3fbfcd309be400460d63a
Request POST /account/update-email | sid=d910ed5ec0e3fbfcd309be400460d63a | origin=http://localhost:5050 | referer=http://localhost:5050/ | authenticated=true
BLOCKED: invalid/missing CSRF token on update-email | sid=d910ed5ec0e3fbfcd309be400460d63a | origin=http://localhost:5050 | referer=http://localhost:5050/
Request GET /account | sid=e9eb7b4fa03171898e53ce7901b39bd8 | origin=none | referer=http://localhost:4001/login | authenticated=true
Request GET /account | sid=e9eb7b4fa03171898e53ce7901b39bd8 | origin=none | referer=http://localhost:4001/login | authenticated=true

```

This log provides detailed information about all the requests.

4 Evidence of Project Management

4.1 Task Allocation

Task	Assignees
CSRF code Section 2.1-2.2 and 3 (Report)	Dilshodbek Niyazimbetov
OAuth Session Fixation code Section 2.3, 4 and 6 (Report)	S M Ihtesham Tahmid
OAuth Session Fixation test Section 2.4-2.8 and 5.2 (Report)	Jun Ching Chai
CSRF test Section 1 and 5.1 (Report)	Gerard Vincent Cabriles

4.2 Effort Estimation and Scheduling

The project had a short two week schedule to complete the project. The team began breaking the work into discrete tasks and estimating the effort each would require.

- Before any development began, the project was broken into its major tasks and each was logged as an issue in GitHub projects with an estimated effort and an assigned owner.
- Recording these estimates up front gave the team a shared understanding of where the heaviest work lay and made it possible to compare estimated effort against actual effort as the project progressed.

The estimates proved to be largely accurate with a few tasks running slightly over.

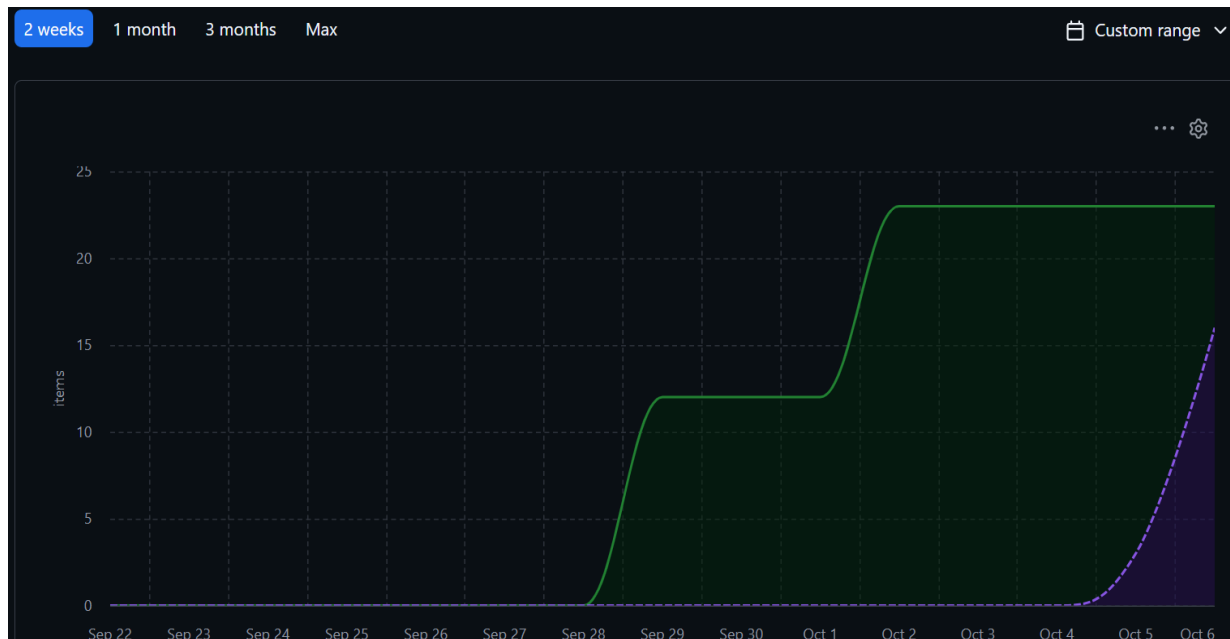
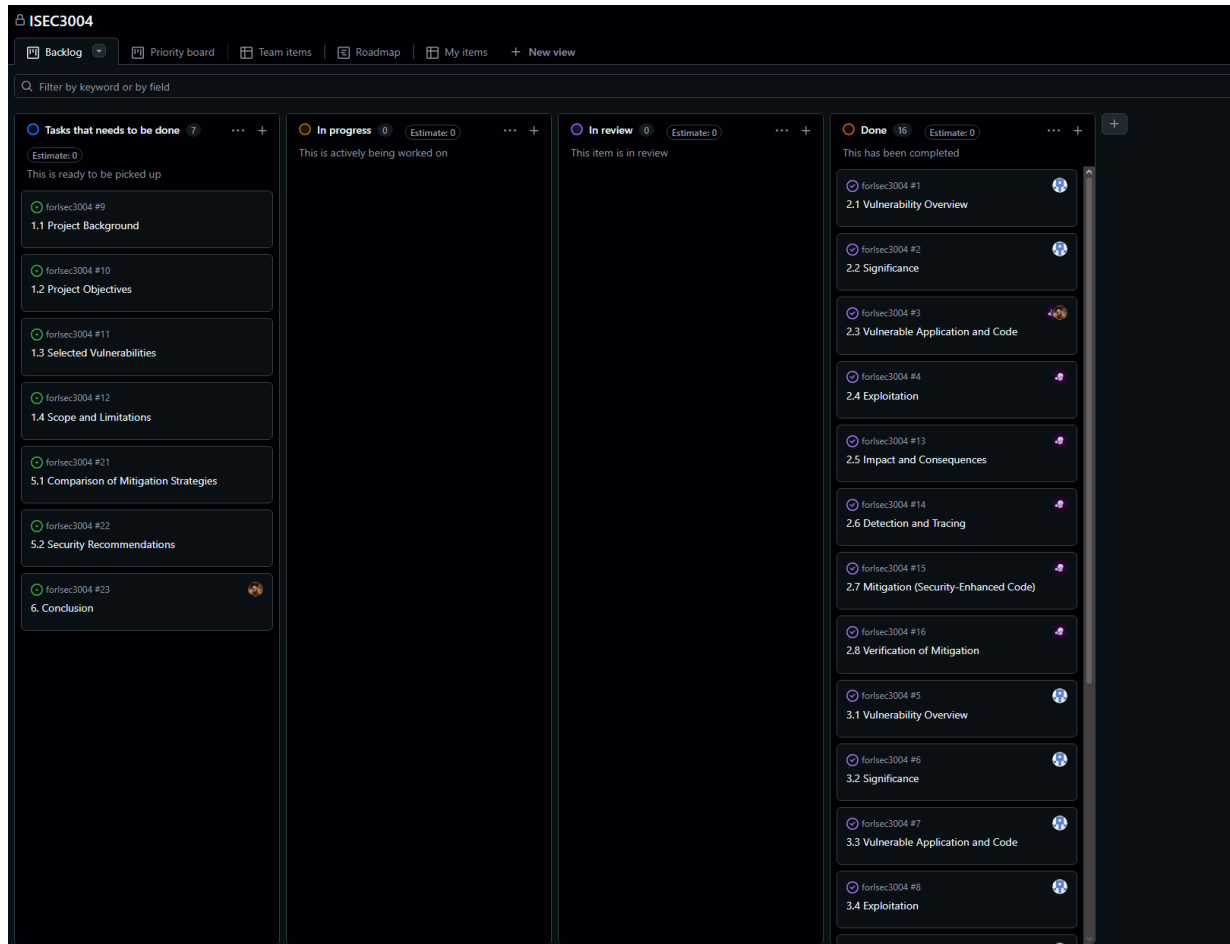
The most demanding task was developing the two vulnerable applications. It took more time as it needed additional effort to make the insecure behaviour both realistic and clearly commented. Report writing went as planned according to the estimates.

Issues							New issue
is:issue state:closed							
☐ Open	2	☒ Closed	25	Author	Labels	Projects	Milestones
							Assignees
							Filter Newest
☐	☒	4.4 Quality Assurance		snipexCS/forlsec3004	albionix121	—	—
☐	☒	4.3 Progress Monitoring		snipexCS/forlsec3004	albionix121	—	—
☐	☒	4.2 Effort estimation and scheduling		snipexCS/forlsec3004	albionix121	—	—
☐	☒	4.1 Task Allocation		snipexCS/forlsec3004	albionix121	—	—
☐	☒	6. Conclusion		snipexCS/forlsec3004	snipexCS	—	—
☐	☒	3.8 Verification of Mitigation		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	3.7 Mitigation (Security-Enhanced Code)		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	3.6 Detection and Tracing		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	3.5 Impact and Consequences		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	2.8 Verification of Mitigation		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	2.7 Mitigation (Security-Enhanced Code)		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	2.6 Detection and Tracing		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	2.5 Impact and Consequences		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	1.4 Scope and Limitations		snipexCS/forlsec3004	snipexCS	—	—
☐	☒	1.3 Selected Vulnerabilities		snipexCS/forlsec3004	snipexCS	—	—
☐	☒	1.2 Project Objectives		snipexCS/forlsec3004	snipexCS	—	—
☐	☒	1.1 Project Background		snipexCS/forlsec3004	snipexCS	—	—
☐	☒	3.4 Exploitation		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	3.3 Vulnerable Application and Code		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	3.2 Significance		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	3.1 Vulnerability Overview		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	2.4 Exploitation		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	2.3 Vulnerable Application and Code		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	2.2 Significance		snipexCS/forlsec3004	snipexCS	—	1
☐	☒	2.1 Vulnerability Overview		snipexCS/forlsec3004	snipexCS	—	1

Figure: Issues Implemented on GitHub Projects

4.3 Progress Monitoring

Progress has been monitored continuously using GitHub projects and the project’s Git history. Each task existed as an issue on a Kanban board with three columns: to-do, in-progress and done. Therefore, the status of every piece of work was visible to the whole team.



4.4 Quality Assurance

Quality assurance was integrated throughout the project rather than treating it as a final step. The team applied the following practices to ensure that the code, exploits, mitigations and tracing evidence were correct and reliable:

- Code Review: All the changes made to the project were shared using GitHub repository and reviewed by team members before being accepted.
- Exploit Validation: Each exploit was run end-to-end to confirm it actually demonstrated the vulnerability.
- Mitigation Verification: The identical attack was re-run against the mitigated version to confirm it was blocked.
- Automated Testing: A headless verification script (verify.js) was written to run full session fixation flow automatically and report exploitation against vulnerable app and secure with mitigated app, allowing the result to be re-checked quickly and consistently.
- Verification of Tracing Evidence: The application logs were reviewed to confirm they accurately recorded the malicious activity.
- Report QA: The report was proof read by multiple members for technical accuracy, structure, and consistency before submission.

5 Discussions & Recommendations

5.1 Comparison of Mitigation Strategies

The two vulnerabilities investigated in this project require different mitigation strategies as they target different aspects of an authenticated web application.

- OAuth Session Fixation exploits weaknesses in session management during the authentication process, whereas CSRF exploits insufficient verification of requests made within an already authenticated session.
- Although both attacks can result in both unauthorized access or actions, the security controls required to mitigate them operate at different stages of the authentication and request process.

For OAuth Session Fixation, the primary mitigation implemented in this project is the regeneration of the session identifier following successful authentication.

- In the vulnerable implementation, an attacker can provide a known session ID through a URL, which is then retained when the victim authenticates.
- As a result, the attacker can reuse the known session ID to access the victim's authenticated session. This behaviour represents a session fixation weakness.

OWASP recommends that a session ID be renewed following a change in privilege level, particularly when a user transitions from an unauthenticated to an authenticated state. Previous session identifiers should be invalidated to prevent their reuse.

The security enhanced implementation developed in this project follows this by generating a new session ID following a successful authentication and invalidating the previous identifier. It also prevents a generic session ID supplied through URL from being trusted by the application.

This is consistent with OWASP guidance, which recommends that applications use strict session management, avoid accepting session IDs that were not generated by the application, and avoid URI parameters as mechanism for accepting session identifiers because they can facilitate session fixation.

5.2 Security Recommendations

Based on the vulnerabilities investigated in this project, the following security recommendations are made to reduce the risk of similar vulnerabilities being exploited in software systems.

- Never accept session IDs or CSRF tokens from user-controlled input.
- Always regenerate session IDs whenever a privilege change occurs.
- Always log and monitor unusual behaviour within the system.
- Educate people to always check links before opening them.

6 Conclusion

This project investigated two significant web application vulnerabilities OAuth session fixation and Cross-site Request Forgery (CSRF).

- For each vulnerability a complete security lifecycle has been provided.
- The security lifecycle includes: a working vulnerable application, a reproducible exploit, detection and tracing of the malicious activities, an effective mitigation and verification that the mitigation prevents the attack.

For OAuth session fixation, the flaw lies in the relying application rather than identity provider.

- The application fails to regenerate the session identifier after a successful OAuth login and trusts a session ID supplied in the URL. Thus allowing the attacker to fixate a known session and takeover the victim's account without ever learning the password.
- The mitigation regenerates the session ID after authentication, binding and verifying the OAuth state parameter and hardening the session cookies.

For CSRF, it has been demonstrated that relying solely on a session cookie to authorise state-changing requests lets any site the victim visits forge those requests on their behalf.

- The vulnerabilities were mitigated by implementing the synchronizer token pattern with a per session token verified on every state changing request. Thus blocking the forged request while leaving legitimate use unaffected.

Beyond the technical findings, the project reinforced two broader security principles:

- Trusted third party components do not absolve an application of its own session management responsibilities.
- Correct authorisation must verify intent, not merely the presence of a valid session.

The project has been delivered as a coordinated team effort, supported by project management allocations, effort estimation, scheduling, progress monitoring and quality assurance that have been managed using GitHub projects and version control.

7 References

OWASP Foundation. (2026). *Session Fixation*.

https://community.owasp.org/attacks/session_fixation

SentinelOne. (2026, April 29). *What Is Session Fixation? How Attackers Hijack User Sessions*.

<https://www.sentinelone.com/cybersecurity-101/cybersecurity/session-fixation>

GeeksforGeeks. (2022, July 31). *Session Fixation Attack*.

<https://www.geeksforgeeks.org/ethical-hacking/session-fixation-attack>

nokia. (2025). *OWASP-CheatSheetSeries/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.md at master · nokia/OWASP-CheatSheetSeries*. GitHub.

https://github.com/nokia/OWASP-CheatSheetSeries/blob/master/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.md

OWASP. (2012). *Cross-Site Request Forgery Prevention · OWASP Cheat Sheet Series*.

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

Detectify. (2016, July 19). *OWASP TOP 10 2013: Cross-site Request Forgery – CSRF*.

<https://blog.detectify.com/best-practices/owasp-top-10-cross-site-request-forgery-8>

Waratek. (n.d.). *Best Practices – Cross-Site Request Forgery (CWE-352)*.

<https://support.waratek.com/knowledge/best-practices-cross-site-request-forgery>

8 Appendix

Link for Project Repository Code:

<https://github.com/snipexCS/forlsec3004>

Link for Project To-Do Board:

<https://github.com/users/snipexCS/projects/1/views/1>